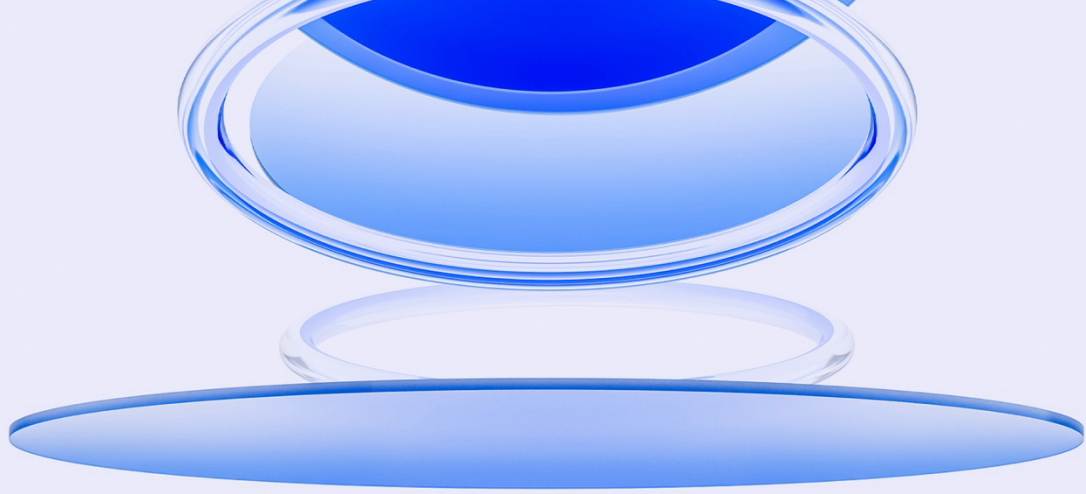




IDENTIFICACIÓN DE VULNERABILIDADES CON ARQUITECTURA AGENTICA

Análisis del ecosistema de plugins Wordpress

FUTURE CYBERSECURITY LAB



ÍNDICE

1. Resumen ejecutivo	¡Error! Marcador no definido.
2. El problema	4
2.1. Las reglas del juego han cambiado	4
2.2. Velocidad de máquina, ritmo humano	4
2.3. La carga cognitiva del triaje	5
2.4. Lo que hace falta	6
3. Enfoque propuesto	6
3.1. Principios de diseño	6
3.2. Modos de análisis	7
3.3. Pipeline de análisis	7
3.4. El conocimiento como guía de validación.....	8
3.5. Arquitectura	8
4. Modelo de resultados y confianza	10
4.1. Por qué clasificar los resultados	10
4.2. Lectura general de los estados.....	10
4.3. De la evidencia a la decisión	10
4.4. Interpretación de los resultados.....	11
5. Evolución respecto a otras arquitecturas	11
6. Resultados de la campaña.....	12
6.1. Aviso legal y de uso responsable	12
6.2. Alcance y entorno de la campaña	12
6.3. Vulnerabilidades con CVE publicado.....	13
6.4. Distribución de los resultados publicados.....	15
7. Divulgación responsable	15
7.1. Principios	15
7.2. Entorno de verificación	16
7.3. Proceso de revisión y coordinación	16
7.4. El papel de la CNA	17
7.5. Qué se publica y qué no	17
7.6. Casos rechazados	18
8. Conclusiones finales	18

1. RESUMEN EJECUTIVO

Esta arquitectura agéntica de identificación y validación de vulnerabilidades propone una forma más precisa de transformar señales de riesgo en evidencia verificada. Frente a los enfoques que generan grandes volúmenes de alertas cuya revisión depende después del criterio manual o humano, este modelo combina análisis automatizado, ejecución en entornos controlados y criterios de validación reproducibles. El objetivo es poder distinguir de una forma más clara y precisa entre hipótesis, señales pendientes y resultados confirmados.

El punto de partida es un problema habitual en seguridad del software. Muchas herramientas identifican patrones potencialmente vulnerables en el código, pero no siempre pueden demostrar si esos patrones son realmente vulnerabilidades. La diferencia entre un patrón sospechoso o potencialmente vulnerable y evidencia puede llevarnos a falsos positivos o poner el foco de atención sobre casos que no sean reales (con la pérdida de tiempo por parte del auditor).

El enfoque propuesto en esta arquitectura propone una diferencia entre la generación de hipótesis y su validación. Primero identifica indicios relevantes y los convierte en escenarios comprobables. Posteriormente, los evalúa en un entorno controlado, y compara el comportamiento esperado con el que realmente se logra (el observado). De este modo, el resultado no se apoya únicamente en la interpretación del código, sino en evidencias recogidas durante la ejecución.

La utilización de modelos de lenguaje se plantea como un apoyo al razonamiento y a la generación de pruebas controladas, no como una sustitución del criterio de validación. El veredicto final se deriva de reglas estables aplicadas sobre lo observado en el entorno controlado de forma dinámica. Esto permite mantener trazabilidad, reducir ambigüedad y conservar niveles diferenciados de confianza cuando la evidencia no es suficiente para confirmar un resultado.

Se ha realizado una campaña sobre un conjunto amplio de plugins de WordPress formado por más de 25.000 plugins. Se ha reportado 321 informes de vulnerabilidades. La CNA de WPScan ha aceptado 261 (81%). En el desarrollo del documento se puede encontrar el número de CVE públicos obtenidos en este proceso (no está cerrado, ya que este tipo de procesos suelen durar meses). Esta validación externa de los casos remitidos refuerza la utilidad del enfoque para filtrar ruido y priorizar evidencias relevantes (identificación estática, trabajada con evidencias obtenidas en entornos dinámicos).

WordPress es una de las plataformas web más utilizadas del mundo¹ y cuenta con un amplio ecosistema formado por miles de plugins, algunos presentes en millones de sitios. Esta gran implantación hace que la seguridad de sus extensiones sea especialmente relevante y refuerza el valor de la campaña de detección de vulnerabilidades que se ha trabajado.

Los resultados publicados incluyen vulnerabilidades con impacto diverso, desde exposiciones de información y deficiencias de control de acceso hasta inyecciones SQL o XSS (Cross-Site Scripting). En conjunto, los componentes afectados alcanzan, prácticamente, el millón de instalaciones activas. Esto muestra la relevancia práctica de auditar también software de menor visibilidad que normalmente queda fuera de revisiones manuales exhaustivas.

La automatización y el uso de una arquitectura agéntica enfocada en un análisis estático y en su posterior comprobación dinámica sobre entornos controlados permite obtener evidencias mucho más justificadas. La propuesta de este sistema es el de disponer de mejor precisión y ser capaz de probar en entornos controlados de forma mucho más ágil y escalable.

2. EL PROBLEMA

2.1. Las reglas del juego han cambiado

El desarrollo de software se ha acelerado de forma notable gracias a la incorporación de la inteligencia artificial. Este aumento en la velocidad de desarrollo ha generado una mayor carga cognitiva que, en buena medida, se ha trasladado a fases posteriores del ciclo de CI/CD. Al mismo tiempo, la creciente dependencia de componentes, librerías y servicios de terceros amplía la superficie de riesgo asociada a código externo sobre el que no siempre existe un control directo. Como resultado, una parte relevante de la exposición técnica de una organización reside en software que no siempre puede revisarse con la profundidad deseable.

La generación asistida por modelos de lenguaje intensifica esta diferencia de ritmo. El código puede producirse e integrarse con una velocidad creciente, pero revisar su comportamiento con criterio, contexto y fiabilidad continúa siendo una tarea especializada. Esta asimetría obliga a replantear cómo se priorizan las señales de riesgo y cómo se decide qué merece atención inmediata.

2.2. Velocidad de máquina/IA Vs Ritmo humano

Las herramientas de seguridad pueden producir alertas de forma casi inmediata, pero comprobar si una señal representa un riesgo real requiere interpretar el contexto, entender las condiciones

¹ <https://w3techs.com/technologies/details/cm-wordpress>

de ejecución y valorar el impacto observable. La capacidad de generar indicios ha crecido mucho más rápido que la capacidad humana para analizarlos con el mismo nivel de precisión.

Este desequilibrio se traduce en colas de triaje cada vez más difíciles de gestionar. Subir umbrales, silenciar reglas, revisar solo muestras o aplazar familias completas de alertas puede reducir el volumen visible, pero no resuelve la cuestión principal: distinguir qué señales justifican una investigación adicional y cuáles no aportan evidencia suficiente.

2.3. La carga cognitiva del triaje

Una alerta rara vez contiene por sí sola toda la información necesaria para tomar una decisión. El auditor debe entender el contexto, hacer la comprobación sobre la condición que ha disparado el caso y, por supuesto, valorar si lo que se observa es una explotación de la vulnerabilidad.

1. ¿Es alcanzable? Existe una señal que nos permite identificar un potencial flujo hacia una vulnerabilidad. Todavía hay que comprobar si las validaciones, la configuración o el propio comportamiento de la aplicación lo hacen efectivo.
2. ¿Qué parte depende de datos externos o no confiables? No todas las entradas tienen el mismo nivel de exposición ni generan el mismo riesgo potencial.
3. ¿Existe un efecto observable (hay explotación)? Llegar a una operación sensible no basta, será necesario comprobar si se produce un resultado medible y relevante en las condiciones evaluadas. En otras palabras, hay que verificar que lo que ocurre en la prueba dinámica es realmente una vulnerabilidad.

Responder a estas preguntas exige conocimiento técnico, contexto del componente y, en muchos casos, una forma controlada de observar el comportamiento real. Ese esfuerzo es difícil de reutilizar, porque cada alerta depende de condiciones particulares de código, configuración, permisos y entorno.

Cuando el volumen supera la capacidad de revisión, el proceso tiende a apoyarse en atajos razonables pero imperfectos:

- Patrones conocidos.
- Severidades estimadas.
- Reglas de descarte.

Esto puede ser operativo a corto plazo, aunque también puede dejar señales relevantes sin validar o transmitir una sensación de seguridad que no siempre está respaldada por evidencia suficiente.

2.4. Lo que hace falta

La necesidad real está en mejorar la calidad de las evidencias asociadas a una vulnerabilidad potencial. De este modo, una señal será útil cuando permite entender qué se ha observado, en qué condiciones y cuál es su nivel de confianza. Ese cambio de enfoque reduce la dependencia de interpretaciones aisladas y ayuda a reservar la revisión humana para los casos con mayor potencial de impacto.

Para ello, cada resultado debe distinguir entre resultados con evidencia suficiente, señales que requieren revisión adicional, superficies evaluadas sin impacto observable y casos que no pudieron analizarse con garantías. Esta clasificación aporta una lectura más útil para priorizar, decidir qué revisar y comunicar los resultados con prudencia, sin convertir una hipótesis en una conclusión definitiva antes de tiempo.

3. ENFOQUE PROPUESTO

3.1. Principios de diseño

La arquitectura propuesta se apoya en un conjunto reducido de principios que orientan todo el proceso. En primer lugar, se separa la generación de hipótesis de su validación. Se exige una evidencia observable antes de confirmar un resultado y se conserva la incertidumbre cuando la prueba no alcanza un umbral suficiente (podría repetirse o entrar una revisión humana para descartar).

- Las señales identificadas en el código se tratan como hipótesis de trabajo, no como conclusiones definitivas.
- La confirmación depende de evidencia reproducible observada durante la ejecución controlada.
- El método de validación se mantiene estable, aunque cambien el lenguaje, el componente o la fuente inicial de la señal.
 - Las señales que no alcanzan evidencia suficiente permanecen visibles para revisión, sin elevarse prematuramente a resultados confirmados.
 - La precisión prevalece sobre el volumen: es preferible obtener menos resultados, pero con una justificación más sólida.
 - El análisis mantiene una postura prudente ante entradas externas y superficies expuestas, sin asumir seguridad ni riesgo sin evidencia suficiente.

3.2. Modos de análisis

La aproximación propuesta puede aplicarse en distintos escenarios de análisis. En todos ellos, el objetivo es el mismo. La conversión de indicios iniciales en evidencias contrastables y la reducción de la dependencia de decisiones basadas únicamente en alertas estáticas.

En un primer escenario, el análisis parte del propio código y del comportamiento del componente para identificar señales que merecen una comprobación posterior.

En otro escenario, una herramienta de análisis estático aporta señales iniciales que se someten después a una validación dinámica. De esta forma, el análisis estático ayuda a priorizar, pero no determina por sí solo el resultado final.

Un tercer escenario combina ambas aproximaciones, es decir, se valida señales procedentes de herramientas externas y, cuando el contexto lo justifica, amplía la revisión a superficies relacionadas. El valor de este enfoque está en ordenar la investigación, no en multiplicar el número de alertas.

En todos los casos, las herramientas automáticas actúan como fuentes de hipótesis. La decisión relevante se reserva para la fase de validación, donde se evalúa si existe evidencia suficiente para confirmar, mantener en revisión o descartar una señal.

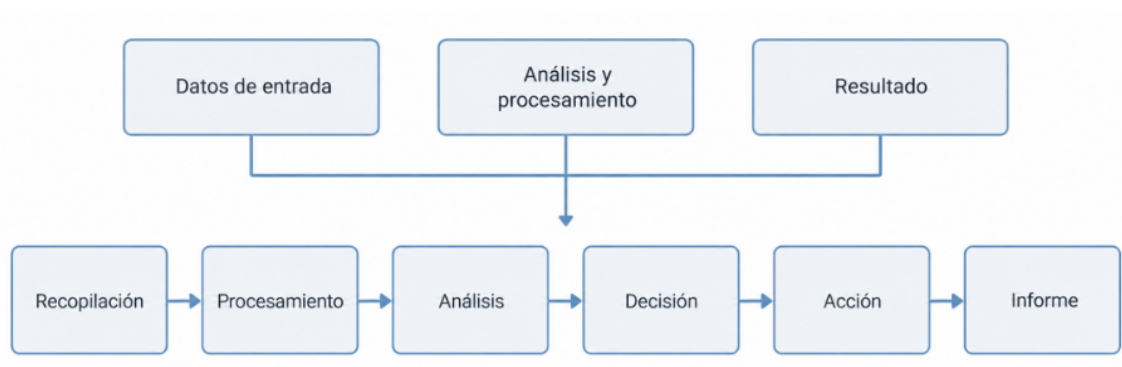


Figura 1. Visión general del flujo de análisis y validación.

3.3. Pipeline de análisis

El proceso se organiza como una validación progresiva:

1. Se identifican señales que pueden merecer atención.
2. Se transforman en hipótesis comprobables.
3. Se contrastan en condiciones controladas para determinar si existe evidencia suficiente.

La confirmación no depende de una única señal ni de una interpretación aislada del código, sino de la convergencia entre contexto, observación y criterios de validación previamente definidos.

Cuando esa evidencia no alcanza el umbral necesario, el resultado conserva la incertidumbre y queda disponible para revisión posterior.

3.4. El conocimiento como guía de validación

Para que el análisis sea consistente, el conocimiento de seguridad se organiza como una guía de validación. En lugar de depender únicamente de alertas aisladas o de decisiones ad hoc, la arquitectura relaciona las familias de riesgo con criterios de comprobación, niveles de evidencia esperados y condiciones que deben observarse antes de considerar confirmado un resultado.

Esta guía permite mantener una cobertura amplia sin exponer el funcionamiento interno del proceso. Las categorías generales de riesgo se tratan de forma homogénea, independientemente del lenguaje o del ecosistema analizado, y se complementan con criterios específicos cuando el contexto técnico lo requiere.

El criterio central es que cada familia de riesgo debe evaluarse a partir de evidencias observables, no solo a partir de coincidencias en el código. Por ejemplo, una señal relacionada con salida hacia el navegador requiere confirmar un efecto real en ese contexto, mientras que una señal asociada a comunicación saliente debe apoyarse en una observación verificable del comportamiento del servicio. La formulación concreta de esas comprobaciones se mantiene en un nivel interno, pero su propósito es común: evitar que una hipótesis se convierta en conclusión sin evidencia suficiente.

Este enfoque también facilita la reutilización del conocimiento sin convertirlo en una receta operativa. La arquitectura conserva patrones de validación, condiciones de aceptación y límites de confianza, pero los aplica de forma controlada y adaptada al contexto de cada componente. Así se mantiene la trazabilidad del razonamiento sin publicar instrucciones detalladas sobre cómo reproducir cada caso.

En conjunto, el conocimiento actúa como un marco de decisión: orienta qué señales merecen atención, qué tipo de evidencia se necesita para elevar la confianza y cuándo conviene conservar una hipótesis para revisión posterior. Esta aproximación reduce la variabilidad entre análisis, mejora la calidad de los resultados y mantiene una separación clara entre el valor metodológico que puede publicarse y los detalles operativos que deben permanecer fuera de una comunicación externa.

3.5. Arquitectura

Para analizar componentes de terceros con garantías, la arquitectura separa las funciones de coordinación del análisis de aquellas relacionadas con la evaluación controlada del componente analizado. Esta separación permite mantener trazabilidad, aislar cada evaluación y evitar que las condiciones de un caso afecten a los demás.

El principio de diseño es más relevante que la implementación concreta: el proceso debe ordenar las hipótesis, conservar el contexto necesario y contrastar el comportamiento observado antes de elevar una señal a resultado confirmado.

Cuando el entorno no permite completar la evaluación con garantías, la arquitectura registra la limitación y mantiene una lectura prudente del resultado, sin transformar una falta de cobertura en una conclusión de seguridad.

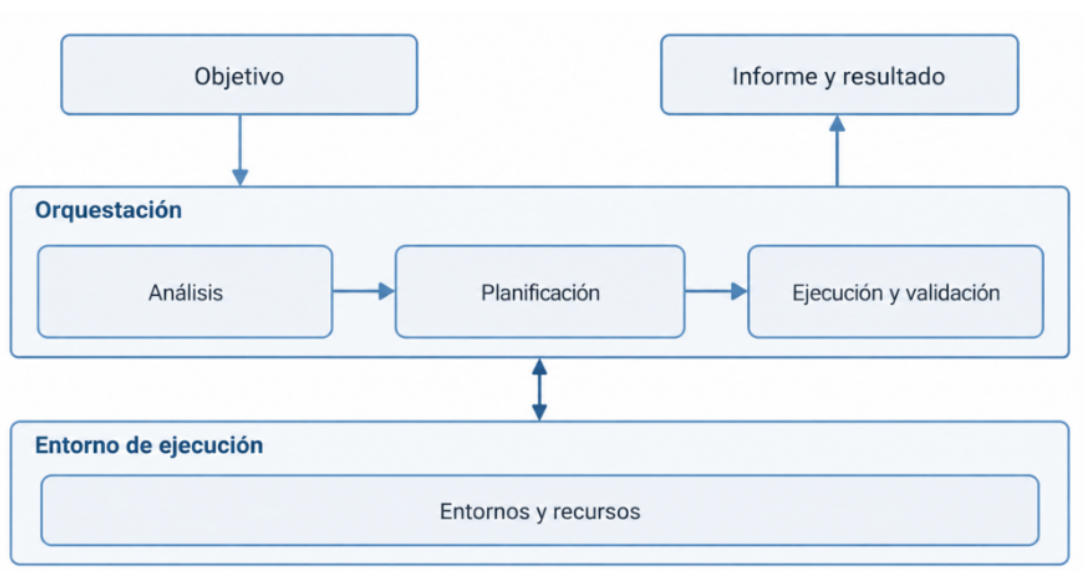


Figura 2. Separación conceptual entre coordinación del análisis y evaluación controlada.

La arquitectura incorpora mecanismos de observación adaptados a distintas familias de riesgo, pero su descripción pública debe mantenerse en un plano general. Su función es contrastar si una hipótesis produce un efecto verificable en condiciones controladas, ya sea en la respuesta de la aplicación, en su comportamiento durante la ejecución o en señales observables asociadas al servicio evaluado.

Estas comprobaciones no se describen como recetas de ejecución, sino como criterios de observación. El objetivo es confirmar que la señal inicial se manifiesta de forma reproducible y relevante, evitando aceptar como evidencia suficiente una mera coincidencia en el código, una respuesta ambigua o una condición que no pueda verificarse en el contexto evaluado.

Cuando una hipótesis requiere varias condiciones previas, la arquitectura conserva el contexto necesario para evaluar la secuencia de forma coherente. Esta capacidad permite razonar sobre escenarios más complejos sin exponer detalles operativos sobre credenciales, pasos intermedios o mecanismos concretos de prueba.

La arquitectura también registra las limitaciones encontradas durante la evaluación. Si una parte del componente no puede analizarse con garantías, el resultado refleja esa falta de cobertura en lugar de convertirla en una conclusión de seguridad. Esta distinción ayuda a interpretar los

resultados con prudencia y evita presentar como descartado aquello que simplemente no pudo comprobarse.

A nivel operativo, la ejecución de campañas se gestiona mediante mecanismos de coordinación que permiten ordenar análisis, supervisar su evolución y preservar la consistencia de los resultados. En una publicación externa, lo relevante no es la implementación concreta de esos mecanismos, sino que el proceso mantiene control sobre cada evaluación y evita mezclar estados o evidencias entre casos.

Cuando una ejecución no produce evidencia suficiente o el entorno no permite completar la evaluación, el caso queda marcado para revisión en lugar de incorporarse como resultado confirmado. Esta política conserva la trazabilidad del análisis y refuerza el principio general del documento: priorizar conclusiones justificadas sobre volumen de resultados.

4. MODELO DE RESULTADOS Y CONFIANZA

4.1. Por qué clasificar los resultados

En una campaña de análisis, no todos los resultados tienen el mismo valor ni grado de certeza. Una señal puede estar plenamente respaldada por evidencia observable, requerir una comprobación adicional o quedar limitada por las condiciones del entorno. Por eso la arquitectura conserva el nivel de confianza alcanzado y el contexto necesario para interpretar cada caso.

Esta clasificación permite diferenciar entre resultados con evidencia suficiente, señales que deben revisarse de nuevo, superficies evaluadas sin impacto observable y casos que no pudieron analizarse con garantías. La utilidad del modelo está en ordenar la revisión y evitar que una hipótesis incompleta se presente como conclusión definitiva.

4.2. Lectura general de los estados

Los estados se agrupan en tres grandes familias que son los resultados confirmados, señales que conservan interés, pero necesitan revisión adicional y análisis que no ofrecen evidencia suficiente para sostener una conclusión. Esta agrupación proporciona una lectura práctica del resultado sin exponer reglas internas, umbrales específicos ni detalles de reproducción (que se podrían disponer en la propia arquitectura agentica).

4.3. De la evidencia a la decisión

La decisión final no depende únicamente de una señal generada durante el análisis. Antes de confirmar un resultado, la arquitectura contrasta esa señal con la evidencia disponible, revisa si el comportamiento observado es coherente con la hipótesis y evita interpretaciones que puedan deberse a coincidencias o pruebas incompletas.

Cuando la evidencia es suficiente, el resultado puede incorporarse al conjunto de resultados confirmados y servir como base para una revisión manual o una divulgación coordinada. Cuando la evidencia queda por debajo de ese umbral, el caso se conserva como señal pendiente o como análisis no concluyente, manteniendo una lectura prudente del alcance real de la evaluación.

4.4. Interpretación de los resultados

La interpretación de los resultados debe separar con claridad lo confirmado, lo pendiente de revisión y lo no evaluado con garantías. Un resultado confirmado indica que existe evidencia suficiente en las condiciones analizadas. Una señal pendiente conserva valor como indicio que requiere más contexto. Una limitación de cobertura no debe entenderse como ausencia demostrada de riesgo.

Esta lectura permite priorizar la revisión humana sin sobredimensionar las conclusiones. La automatización aporta escala y consistencia, pero la comunicación de los resultados debe mantener una distinción prudente entre evidencia suficiente, hipótesis razonables y casos en los que el análisis no permitió alcanzar una conclusión sólida.

5. EVOLUCIÓN RESPECTO A OTRAS ARQUITECTURAS

La Arquitectura de validación dinámica de vulnerabilidades conserva la idea central de utilizar análisis automatizado y modelos de lenguaje para apoyar la comprobación de sospechas de vulnerabilidad. Sin embargo, la aproximación propuesta en este documento no se plantea como una simple reimplementación de ese modelo, sino como una evolución orientada a campañas reales de auditoría, con mayor énfasis en la escala, la trazabilidad, la priorización y la validación externa de los resultados.

En **VulnSage**, el foco principal se sitúa en comprobar si una sospecha previa puede convertirse en una evidencia suficiente para confirmar o descartar una alerta. La arquitectura propuesta amplía ese planteamiento. Además de partir de señales generadas por herramientas externas, puede organizar el análisis como un proceso más amplio de identificación, contextualización y validación progresiva. Este cambio desplaza el objetivo desde la confirmación puntual de una alerta hacia la construcción de una lectura más completa del componente analizado.

La diferencia principal está en el papel que desempeña la validación dentro del proceso. En lugar de tratar el resultado como una decisión binaria, la arquitectura distingue entre resultados confirmados, señales que requieren revisión adicional y casos en los que no existe evidencia suficiente como para defender una conclusión. Esta lectura graduada permite priorizar mejor el trabajo humano y comunicar los resultados con mayor prudencia.

También cambia la orientación práctica del enfoque. VulnSage se presenta principalmente como una propuesta de validación automatizada en un contexto experimental, mientras que la arquitectura descrita aquí se orienta a campañas de análisis a escala, donde importa tanto la

calidad de la evidencia como la capacidad de gestionar grandes volúmenes de componentes, filtrar ruido y preparar resultados para procesos de divulgación coordinada.

Resumiendo, la aportación de la arquitectura no consiste únicamente en automatizar la comprobación de una sospecha, sino en integrar esa validación dentro de un marco más amplio de auditoría: desde la generación inicial de señales hasta la priorización, revisión manual, coordinación externa y publicación responsable de los resultados confirmados.

6. RESULTADOS DE LA CAMPAÑA

6.1. Aviso legal y de uso responsable

Este documento es elaborado y publicado por Telefónica Innovación Digital (TID), a través de su Future Cybersecurity Lab, como autor y entidad promotora del contenido. La información contenida en este documento se proporciona exclusivamente con fines técnicos, defensivos y de investigación en seguridad. Las comprobaciones descritas se realizaron sobre código disponible públicamente y en entornos propios, controlados y aislados, sin efectuar pruebas contra sistemas de terceros ni instalaciones ajenas. El documento no incluye pruebas de concepto completas, instrucciones de explotación ni detalles operativos destinados a facilitar la reproducción no autorizada de vulnerabilidades.

Los resultados publicados se limitan a casos ya divulgados oficialmente a través de los canales de coordinación correspondientes. La aceptación o publicación de vulnerabilidades por parte de una CNA debe entenderse únicamente como validación de los casos concretos admitidos, y no como certificación, aprobación o aval del método completo, de la arquitectura o de sus componentes internos. Las cifras de instalaciones activas que se citan en este documento son una métrica pública, estimada y dinámica, obtenida de las fuentes oficiales en la fecha de consulta indicada; no identifican instalaciones concretas ni implican que el número de sistemas efectivamente vulnerables a fecha de lectura coincida con la cifra publicada. Cualquier uso de la información aquí descrita debe realizarse conforme a la legislación aplicable, las políticas de divulgación responsable y contando siempre con la autorización correspondiente.

6.2. Alcance y entorno de la campaña

La campaña descrita en este apartado se realizó exclusivamente sobre código disponible públicamente y en entornos propios, controlados y aislados. Las comprobaciones no se ejecutaron contra sistemas de terceros, instalaciones ajenas ni servicios en producción fuera del ámbito del laboratorio. Esta delimitación es relevante para interpretar correctamente los resultados. El objetivo fue transformar señales técnicas en evidencias reproducibles bajo condiciones controladas, no probar componentes desplegados por terceros ni evaluar infraestructuras externas.

El documento publica únicamente casos cuya divulgación ya ha sido realizada oficialmente a través de los canales de coordinación correspondientes. No se incluyen casos pendientes de revisión, componentes todavía bajo coordinación ni información operativa que permita reproducir vulnerabilidades no divulgadas. Los resultados que permanecen en tramitación se tratan, como máximo, de forma agregada y estadística, sin identificar componentes ni aportar detalles técnicos sensibles.

La aceptación o publicación de una vulnerabilidad por parte de WPScan debe entenderse únicamente como revisión y admisión de los informes concretos remitidos. En ese proceso, WPScan evalúa cada caso, decide si resulta admisible dentro de sus criterios, coordina con el desarrollador y publica la ficha cuando procede (proceso de disclose). Esa intervención no constituye una certificación, aprobación o aval del método completo, de la arquitectura, de sus componentes internos, de la tasa global de precisión ni del proceso utilizado para generar candidatos antes de la revisión manual y la comunicación formal.

6.3. Vulnerabilidades con CVE publicado

La campaña se ejecutó sobre el repositorio público de plugins de WordPress, con un total de 25.537 plugins analizados. De acuerdo con el modelo de resultados y confianza descrito en el apartado 4, la revisión manual se centró en los casos con mayor respaldo de evidencia. De ese conjunto, el análisis manual documentó y remitió a la CNA 321 informes.

De esos 321 informes, WPScan rechazó 60 por motivos diversos (existencia de un identificador previo o discrepancia sobre si era explotable) y admitió 261, lo que supone una tasa de aceptación del 81 %. Se trata de una validación externa e independiente del proyecto. Las vulnerabilidades superan el filtro técnico de un tercero antes de recibir un identificador.

ETAPA	TOTAL
Plugins analizados	25.537
Informes reportados a la CNA	321
Informes aceptados por la CNA	261
CVE asignados	54
CVE publicados	40

Tabla 1: Datos generales de la campaña

Conviene señalar, además, que estas cifras describen únicamente los casos con mayor respaldo de evidencia. La campaña dejó también un conjunto amplio de señales que no alcanzaron el umbral de confirmación y que permanecen pendientes de revisión humana. Ese conjunto no se ha explorado en detalle y no interviene en ninguna de las cifras anteriores, por lo que los resultados de la campaña deben entenderse como una estimación conservadora del potencial identificado.

La tabla siguiente recoge los identificadores ya publicados a fecha de publicación de este documento. Los CVE asignados cuya divulgación sigue en coordinación con el desarrollador no se detallan aquí, en coherencia con el proceso descrito en el apartado 7, se incorporarán a esta tabla en la actualización posterior a su publicación.

CVE	PLUGIN	INSTALACIONES ACTIVAS	CWE	CVSS	Ficha
CVE-2026-13402	Royal Elementor Addons	600.000+	CWE-200	5,3 (media)	Ficha WPScan
CVE-2026-92435	Mailchimp for WooCommerce	200.000+	CWE-862	5,3 (media)	Ficha WPScan
CVE-2026-10082	Advanced Ads	100.000+	CWE-79	6,8 (media)	Ficha WPScan
CVE-2026-12512	Quotes Llama	1.000+	CWE-89	8,6 (alta)	Ficha WPScan
CVE-2026-16257	Arvow AI SEO Writer	1.000+	CWE-287	8,2 (alta)	Ficha WPScan
CVE-2026-15236	Gallery for Google Photos	1.000+	CWE-200	7,5 (alta)	Ficha WPScan
CVE-2026-16273	Narrative Publisher	1.000+	CWE-79	6,8 (media)	Ficha WPScan
CVE-2026-16942	WP Custom HTML Pages	1.000+	CWE-79	6,8 (media)	Ficha WPScan
CVE-2026-16539	SM Page Duplicator	1.000+	CWE-89	6,8 (media)	Ficha WPScan
CVE-2026-92430	Rede Itau for WooCommerce	1.000+	CWE-862	5,3 (media)	Ficha WPScan
CVE-2026-16536	Simple Google Calendar Outlook Events Widget	1.000+	CWE-918	5,3 (media)	Ficha WPScan
CVE-2026-86785	Social Commerce for WooCommerce	1.000+	CWE-862	5,3 (media)	Ficha WPScan
CVE-2026-15246	RealHomes Memberships	1.000+	CWE-65	4,3 (media)	Ficha WPScan
CVE-2026-11872	Clever Mega Menu for Visual Composer	1.000+	CWE-284	4,3 (media)	Ficha WPScan
CVE-2026-11882	Builderall for WordPress	1.000+	CWE-284	3,7 (baja)	Ficha WPScan
CVE-2026-77003	Content Mask	1.000+	CWE-269	2,7 (baja)	Ficha WPScan
CVE-2026-92996	Verge3D	400+	CWE-862	5,3 (media)	Ficha WPScan
CVE-2026-16947	Total Processing Card Payments for WooCommerce	100+	CWE-918	9,1 (crítica)	Ficha WPScan
CVE-2026-81737	FAQ Builder AYS	100+	CWE-79	8,8 (alta)	Ficha WPScan
CVE-2026-74933	GenieWords	100+	CWE-79	8,8 (alta)	Ficha WPScan
CVE-2026-87842	Zonify	100+	CWE-200	7,5 (alta)	Ficha WPScan
CVE-2026-88788	Text Styler	100+	CWE-79	6,8 (media)	Ficha WPScan
CVE-2026-87839	Tripzzy	100+	CWE-284	7,5 (alta)	Ficha WPScan
CVE-2026-87840	Tripzzy	100+	CWE-284	5,3 (media)	Ficha WPScan
CVE-2026-94270	Deema Payment Gateway	100+	CWE-287	5,3 (media)	Ficha WPScan
CVE-2026-15958	Easy Dropbox Integration	90+	CWE-862	9,3 (crítica)	Ficha WPScan
CVE-2026-77012	Icollect	80+	CWE-918	9,3 (crítica)	Ficha WPScan
CVE-2026-16061	Rest Routes	80+	CWE-89	8,6 (alta)	Ficha WPScan
CVE-2026-18395	Child Pages Card	80+	CWE-79	6,8 (media)	Ficha WPScan
CVE-2026-77013	Icollect	80+	CWE-862	5,3 (media)	Ficha WPScan
CVE-2026-86801	To Do List Member	60+	CWE-306	8,8 (alta)	Ficha WPScan
CVE-2026-86802	To Do List Member	60+	CWE-345	3,7 (baja)	Ficha WPScan
CVE-2026-81742	BE REST Endpoints	50+	CWE-79	8,8 (alta)	Ficha WPScan
CVE-2026-87786	Dewa Kirim	50+	CWE-79	8,8 (alta)	Ficha WPScan
CVE-2026-16571	phone2app	30+	CWE-79	8,8 (alta)	Ficha WPScan
CVE-2026-16572	LogMyTrip	30+	CWE-89	8,6 (alta)	Ficha WPScan
CVE-2026-92404	MgoSync	20+	CWE-200	7,5 (alta)	Ficha WPScan
CVE-2026-93000	SPS-Suite	20+	CWE-89	6,8 (media)	Ficha WPScan

CVE-2026-88792	Dictionary	10+	CWE-79	8,8 (alta)	Ficha WPSan
CVE-2026-86790	WP Highlight Box	N/A	CWE-79	6,8 (alta)	Ficha WPSan

Tabla 2: CVEs publicados

6.4. Distribución de los resultados publicados

- **Por severidad.** Las vulnerabilidades más graves comparten un patrón que son los componentes que exponen funcionalidades sensibles, como acceso a almacenamiento externo, tráfico saliente del servidor o gestión de pasarelas de pago, a través de puntos de entrada con controles insuficientes.
- **Por tipología.** Las vulnerabilidades detectadas se concentran en un número reducido de categorías: inyección SQL, XSS almacenado, SSRF y deficiencias en el control de acceso, junto a algunos casos de exposición de información sensible. El aspecto más relevante de esta distribución es que se trata de vulnerabilidades ampliamente documentadas y con medidas de mitigación conocidas que, aun así, continúan en el repositorio oficial. Este fenómeno subraya la utilidad de los sistemas automatizados capaces de operar a escala, dado que la auditoría manual resulta inviable para el volumen completo de código disponible.
- **Por nivel de exposición.** Los casos confirmados se reparten entre unos pocos plugins con mayor exposición, que concentran la mayor parte de las instalaciones afectadas, y muchos plugins con uso minoritario. Este patrón resulta significativo desde una perspectiva puramente defensiva: los componentes con baja cuota de usuarios suelen contar con menos recursos de mantenimiento y quedan habitualmente fuera de las auditorías manuales tradicionales, precisamente porque estas priorizan el software más extendido. La automatización permite extender la cobertura de revisión a ese segmento del ecosistema, reforzando así la seguridad general del repositorio en beneficio de todos sus usuarios, con independencia del volumen de instalaciones de cada componente.

7. DIVULGACIÓN RESPONSABLE

7.1. Principios

El proyecto se rige por tres principios que condicionan todas las decisiones operativas descritas a continuación:

- **Ningún sistema de terceros es objeto de pruebas.** La verificación de una vulnerabilidad requiere comprobar su comportamiento en un entorno controlado. Realizar esa comprobación contra una instalación ajena sin autorización expresa puede ser ilegal o no autorizado según la jurisdicción y el contexto. Toda comprobación se realiza en entornos propios, aislados y desechables.

- **El desarrollador dispone de margen antes que el público.** La publicación de detalle técnico solo aporta valor defensivo cuando existe una corrección disponible, es decir, una actualización (un parche de seguridad) de la aplicación o una vía clara de mitigación. Hasta entonces, adelantar información podría incrementar el riesgo de explotación.
- **Se reporta lo verificado.** Un resultado automático no se traslada a un tercero hasta que un analista lo ha reproducido y documentado. Enviar hipótesis sin comprobar traslada el coste de verificación a la CNA primero y al desarrollador después, además de degradar la señal del canal de comunicación.

7.2. Entorno de verificación

Cada candidato se comprueba en un entorno de laboratorio propio, aislado y desechable, preparado para evaluar únicamente el caso analizado. Esta aproximación permite observar el comportamiento del componente en condiciones controladas y evita que el estado de una evaluación influya en la siguiente.

El laboratorio aporta una validación que el análisis estático no ofrece por sí solo: permite comprobar si la condición señalada se manifiesta de forma observable, qué contexto mínimo requiere y si existe impacto suficiente para justificar su comunicación a un tercero. Esa evidencia diferencia un resultado confirmado de una alerta pendiente de revisión.

7.3. Proceso de revisión y coordinación

1. **Recepción del candidato.** La aproximación propuesta clasifica el resultado dentro del conjunto de confianza correspondiente. Los candidatos con mayor respaldo entran en la cola de revisión manual.
2. **Revisión del informe y del código.** El auditor contrasta la señal con el código fuente del componente, revisa el contexto técnico y determina si existen condiciones suficientes para continuar la validación.
3. **Validación en laboratorio.** El caso se evalúa en un entorno controlado de forma automática por parte de la arquitectura agéntica para comprobar si la condición señalada produce un efecto observable y reproducible. Si la evidencia no alcanza el umbral necesario, el caso se mantiene como no concluyente o se descarta de forma prudente.
4. **Caracterización.** Del caso confirmado se determinan versiones afectadas, privilegio mínimo requerido, impacto y categoría CWE aplicable.
5. **Reporte a la CNA.** El caso se traslada a WPScan, CNA de referencia del ecosistema WordPress, a través de su formulario de envío. El informe incluye la información

necesaria para que la CNA evalúe el caso, coordine con el desarrollador y, si procede, gestione la asignación del identificador CVE.

6. **Coordinación y espera.** El caso permanece en tramitación bajo control de la CNA. Durante esta fase no se difunde información alguna sobre el resultado.
7. **Publicación.** Una vez la CNA publica la ficha, el caso puede referenciarse públicamente e incorporarse a la Tabla 6.2 en la siguiente actualización de este documento.

7.4. El papel de la CNA

La gestión de los informes se canaliza íntegramente a través de WPScan en lugar de notificar los fallos directamente a cada desarrollador. Esta decisión responde principalmente a razones de escalabilidad, eficacia operativa y coherencia en la coordinación. En un ecosistema tan amplio y diverso como el de WordPress, los canales de comunicación, los procedimientos de recepción y los tiempos de respuesta pueden variar de forma significativa entre proyectos. Centralizar la comunicación a través de una CNA permite ordenar ese proceso, reducir fricción operativa y mantener una gestión homogénea de los casos reportados.

Delegar esta tarea en la CNA aporta tres ventajas estructurales: garantiza un canal de comunicación institucional que los desarrolladores suelen atender, aplica criterios de admisibilidad y severidad estandarizados, y proporciona una validación independiente previa a la asignación del CVE.

Como contrapartida, ceder la coordinación implica asumir los plazos del evaluador. El proyecto no impone ventanas de divulgación unilaterales (como políticas estrictas de publicación a los N días), sino que se adhiere a los tiempos de la CNA, reteniendo cualquier detalle técnico mientras el expediente permanezca abierto. En consecuencia, un volumen significativo de vulnerabilidades confirmadas se encuentra temporalmente a la espera de identificador, un retraso que se asume como coste inherente a un proceso ordenado.

7.5. Qué se publica y qué no

Este documento detalla exclusivamente aquellas vulnerabilidades cuya ficha técnica ya ha sido publicada de manera oficial por la CNA. Los casos que cuentan con identificador asignado, pero que aún se encuentran en fase de divulgación coordinada, se reportan únicamente a nivel estadístico, omitiendo el nombre del componente afectado y cualquier detalle que pueda facilitar su reproducción antes de que las correcciones sean públicas. Estos detalles se incorporarán a la tabla correspondiente una vez finalizado el proceso de coordinación.

Adicionalmente, bajo el principio de no facilitar reutilización indebida de la información, no se publican pruebas de concepto completas ni fragmentos de código preparados para reproducir el caso fuera de un contexto autorizado. La descripción proporcionada mantiene únicamente la

granularidad técnica necesaria para que un administrador comprenda el riesgo, priorice la actualización y adopte medidas de mitigación cuando correspondan.

7.6. Casos rechazados

Si un informe es rechazado por la CNA, se descarta su publicación por vías alternativas. Cuando la negativa obedece a la existencia de un identificador previo, el caso se archiva como duplicado. Si existe una discrepancia técnica sobre la posibilidad de reproducir el comportamiento en un entorno realista, prevalece el criterio de la CNA y el caso se cierra sin difusión pública.

Esta política de contención se aplica de forma estricta, incluso cuando el equipo investigador considera que la señal mantiene interés técnico. Buscar canales de divulgación paralelos tras un rechazo institucional podría erosionar la relación de confianza con la entidad responsable de evaluar y coordinar el flujo de trabajo.

8. CONCLUSIONES FINALES

La Arquitectura agéntica demuestra que la automatización puede aportar más valor cuando se orienta a producir evidencia fiable, y no simplemente a incrementar el número de alertas. Al combinar análisis automatizado, evaluación controlada y criterios de confianza, el enfoque permite concentrar la revisión humana en los casos con mayor respaldo y comunicar los resultados con mayor prudencia. La campaña descrita muestra que este modelo puede escalar la auditoría de ecosistemas amplios, reducir ruido y apoyar procesos de divulgación responsable sin perder trazabilidad ni rigor.

